

Machine Learning-Based Portfolio Management Strategies

Domenico De Prosperis, Annalisa Conti, Claudio Cappuccini,
Alessandro Mondello

Giugno 2026



Published by **Starting Finance Club Sapienza Area Trading**

Enrico Marra (Head Area Trading)
Gianandrea Parente (Vice-Head Area Trading)

Table of Contents

2	Introduction	3
3	Machine Learning for Portfolio Management	5
3.1	Definition and Application Areas	5
3.1.1	ML Techniques Applicable to a Securities Portfolio	5
3.1.2	Why Use Machine Learning Instead of Classical Models	6
3.2	Portfolio Strategies: Classical Approaches	7
4	Development of the Quantitative Portfolio Management System	8
4.3	System Architecture and Configuration	8
4.3.1	Investable Universe and Survivorship-Bias Prevention	8
4.3.2	Transaction-Cost Modeling	8
4.4	Data Processing and Feature Engineering	8
4.4.1	Technical and Momentum Domain	8
4.4.2	Macroeconomic Domain	9
4.5	Formulation of the Dynamic Target	9
4.6	Predictive Engine, Hyperparameter Optimization, and Walk-Forward Validation	9
4.6.1	Hyperparameter Optimization	10
4.6.2	Walk-Forward Methodology	10
4.7	Backtesting Engine and Risk Parity	11
4.7.1	Risk-Parity Allocation	11
4.7.2	Drift and Rebalancing Management	11
4.8	Evaluation Metrics and Graphical Output	11
5	Analysis and Discussion of Results	13
5.1	Global Comparative Analysis	13
5.2	Equity Curve Comparison	13
5.3	Comparative Annual Returns	14
5.4	Risk-Return Profile	15
5.5	Distribution of Daily Returns	16
6	Economic Interpretation of Results	17
7	Critical Discussion and Limitations	17
8	Conclusions	18

Introduction

The technological transformation of financial markets has profoundly changed the way data are analyzed, investment decisions are made, and portfolio management strategies are built. The exponential growth in computational power, the availability of large amounts of financial and macroeconomic data, and the development of advanced algorithms have fostered the spread of increasingly sophisticated quantitative approaches, grounded in the use of Machine Learning.

In today’s financial environment, traditional econometric models show increasingly evident limitations. Linear regressions and classical asset-pricing frameworks often assume stable and linear relationships between economic variables and future returns—assumptions that rarely hold in real markets, which are instead characterized by high volatility, non-linear dynamics, and continuous structural change. In this scenario, Machine Learning represents a significant evolution, as it allows complex relationships to be learned directly from data without imposing a predefined functional form.

The aim of this dissertation is to analyze the contribution of Machine Learning to quantitative portfolio management, both from a theoretical perspective and through the development of a proprietary operational system for Quantitative Portfolio Management. In particular, the analysis focuses on the application of tree-based and ensemble models, such as Random Forest and XGBoost, integrated with Risk Parity allocation methodologies and advanced temporal validation techniques.

The first part examines the main Machine Learning techniques applicable to quantitative finance, highlighting their characteristics, advantages, and operational limitations. A critical comparison with traditional models is also developed, focusing on the issues of market non-linearity, overfitting, high-dimensional predictors, and adaptability to shifts in financial regimes. Subsequently, some of the most important portfolio management strategies in the literature are analyzed, including Markowitz’s Mean-Variance Optimization, Fama and French’s Factor Investing, and Statistical Arbitrage strategies.

The second part of the dissertation is dedicated to the design and implementation of the quantitative system. The developed framework uses a multi-asset universe including equities, bonds, and ETFs, also incorporating delisted and distressed assets to mitigate survivorship bias. The system integrates more than fifty technical and macroeconomic features, creating a predictive environment capable of contextualizing asset behavior under different market conditions.

One of the project’s central aspects concerns the definition of a dynamic target based on rolling volatility, designed to adapt forecasting thresholds to the different risk characteristics of individual assets. Model training is carried out through a walk-forward methodology with an expanding window, a solution that makes it possible to realistically simulate the operating conditions of a professional manager while avoiding data leakage.

Finally, the third part of the dissertation is dedicated to presenting and discussing the empirical results obtained from the developed framework. The strategy’s performance is analyzed through risk-return metrics, out-of-sample robustness assessments, and comparisons with traditional capital allocation approaches, in order to verify the system’s actual ability to generate value under different market conditions.

The dissertation therefore aims to show that Machine Learning, when integrated with rigorous validation methodologies, risk management, and realistic backtesting, can be an extremely effective tool for building advanced quantitative investment systems, while

still leaving open fundamental questions regarding model interpretability, performance stability, and the computational sustainability of the implemented strategies.

Machine Learning for Portfolio Management

3.1 Definition and Application Areas

Machine Learning (ML) is a set of statistical and computational techniques that enable a system to learn a configuration or pattern from historical data, without having been explicitly programmed to recognize it.

In the context of portfolio management, this means building models capable of identifying relationships among market variables such as prices, returns, volumes, macroeconomic indicators, and future asset returns, dynamically adapting to changing market conditions. Unlike a classic econometric model, where the relationships among variables are specified a priori, an ML model learns these relationships directly from data, capturing non-linear dependencies and complex interactions that traditional models would not be able to detect.

3.1.1 ML Techniques Applicable to a Securities Portfolio

ML techniques applicable to portfolio management differ in complexity, flexibility, and robustness. The choice of one model over another depends on the dataset, the investment horizon, and the specific goals of the strategy, but the literature provides clear guidelines on when to prefer one approach over another.

The main techniques are:

1. **Decision Trees:** a decision tree splits the dataset into homogeneous subgroups until a stopping threshold is reached, expressed by critical numerical values, thereby building a hierarchical structure.

They are useful in finance because they are easy to visualize and can be applied to both numeric and categorical data. They naturally capture non-linear interactions among variables without assuming a predefined functional form; for example, a momentum signal—a technical indicator that measures the strength and speed of an asset’s price changes—may only be meaningful when volatility is low.

A limitation is that a single deep tree easily overfits and exhibits high variance, which is why it is almost always pruned and used as a base learner for subsequent methods.

2. **Random Forest:** it addresses the single-tree problem by building an ensemble of trees, each trained on a random subsample of observations and features, and then aggregating their predictions via an arithmetic or weighted average.

This mechanism, called *bagging*, reduces the model’s variance while keeping bias low: the idiosyncratic noise of each individual tree is averaged out through aggregation, allowing the stable common signal to emerge.

The study by **Krauss et al. (2017)**, titled “*Deep neural networks, gradient-boosted trees, random forests*”, identifies it among the best-performing models for statistical arbitrage on the S&P 500, thanks to its robustness in the presence of many noisy predictors.

3. **Gradient Boosting (XGBoost, LightGBM):** unlike random forests, it builds trees sequentially: each new tree corrects the errors of the previous one, focusing on the hardest cases.

The result is a highly accurate model, but it requires careful regularization to avoid

overfitting.

The study by **Gu et al. (2020)**, "*Empirical Asset Pricing via Machine Learning*", shows that gradient boosting produces some of the best predictions of cross-sectional returns, striking a good balance between flexibility and generalization.

4. **Neural Networks:** the most flexible models. Organized into layers of neurons with non-linear activation functions, neural networks can approximate arbitrarily complex functions, which is why they tend to perform best with more than 100,000 observations.

In finance, different architectures are used depending on the application context:

- Feedforward (MLP): for return forecasts derived from tabular variables;
- LSTM (Long Short-Term Memory): recurrent networks designed to capture temporal dependencies in time series, useful when the sequence of events matters;
- Transformer: a more recent architecture, originally developed for natural language, increasingly applied to financial time series due to its ability to capture long-range relationships.

The same study by **Gu et al. (2020)** shows that deep neural networks produce better overall out-of-sample R^2 results (i.e., predictive ability on previously unseen data), at the cost of higher computational complexity and lower interpretability.

5. **Regularized Regression:** although technically linear models, they provide a natural bridge between classical econometrics and ML.

Regularization penalizes model complexity, discouraging the learning of noise: LASO (Least Absolute Shrinkage and Selection Operator) forces some coefficients exactly to zero, automatically selecting the most relevant variables, while Ridge Regression shrinks the magnitude of all coefficients without setting them to zero.

Ensemble methods such as random forests and gradient boosting offer the best trade-off between performance and robustness, with less need for tuning than neural networks. Neural networks become preferable when the dataset is very large and one wants to capture complex temporal dependencies.

Regularized regressions remain useful as a benchmark for evaluating performance and changes over time.

3.1.2 Why Use Machine Learning Instead of Classical Models

The study by **Gu, Kelly, and Xiu (2020)**, titled "*Empirical Asset Pricing via Machine Learning*" [3], a pioneering work in applying ML to finance, shows that when both approaches are applied to the same dataset of U.S. equities, ML models produce significantly more accurate return predictions, whereas linear models almost completely fail.

There are three main reasons for the failure of linear regression in portfolio management:

1. **Non-linearity of financial markets:** an intrinsic feature of markets is that they move without following stable linear relationships.

A linear regression model assumes that the relationship between an observed variable and future returns is constant and follows a precise form; in reality, this assumption is almost always violated.

ML, through decision trees and algorithms able to identify complex interactions among financial variables without assuming a predefined functional form, improves predictive ability in high-volatility settings and with heterogeneous data.

2. **High dimensionality of predictors:** when the number of predictors is large relative to the number of observations, classical models suffer from overfitting, i.e., they learn the training data perfectly, including incorrect or irrelevant background details (so-called noise). This yields excellent in-sample performance but poor results on unseen data, because the model cannot generalize the previously learned signal. ML is not immune to overfitting, but models such as Random Forest and Gradient Boosting can mitigate it by building multiple decision trees and aggregating their outputs to make accurate predictions or correct decisions on unseen data.
3. **Adaptability to changes over time:** running a linear regression on a historical period produces coefficients that are fixed over time, implicitly assuming that the market structure remains stable after the model is estimated. This is unreliable because when markets undergo structural changes, a static model will continue to generate signals based on an obsolete structure, committing a systematic and persistent error precisely when mistakes are most costly. An ML model retrained periodically updates its parameters by incorporating new information, tracking changes in the market.

3.2 Portfolio Strategies: Classical Approaches

The finance literature offers several portfolio management strategies on which ML can be grafted, or against which it can be evaluated as an alternative. The main ones are:

- **Mean-Variance Optimization** (*Markowitz, 1952*): a classic reference framework. Given a set of assets, it seeks the combination of weights that maximizes expected return for a given level of risk, constructing the so-called efficient frontier. ML does not replace this framework; rather, it integrates naturally by improving the input estimates—expected returns and the covariance matrix—which in the classical model are notoriously sensitive to estimation error and can produce unstable portfolios. Replacing these estimates with forecasts generated by ML models significantly reduces this problem while preserving the theoretical soundness of the Markowitz framework.
- **Factor Investing** (*Fama & French, 1993*): the three-factor model—*market*, *size (SMB)*, and *value (HML)*—explains cross-sectional returns (i.e., at a given point in time) through exposures to systematic risk factors. A portfolio strategy can be built by selectively tilting toward these risk premia. The study by **Gu et al. (2020)** implicitly extends this framework, showing that ML can identify non-linear and dynamic factor relationships that the linear Fama–French model cannot capture, suggesting that risk premia vary over time in complex ways.
- **Statistical Arbitrage.** As shown by *Krauss et al. (2017)*, it consists of exploiting temporary price deviations and discrepancies from the historical relationship among securities. ML identifies these opportunities more precisely than classical cointegration-based methods, dynamically adapting to changes in the structure of relationships among assets.

Development of the Quantitative Portfolio Management System

4.3 System Architecture and Configuration

The system integrates non-linear predictive models (Random Forest, XGBoost, and Ensemble logic) with capital allocation techniques based on risk parity (Risk Parity)¹. Particular emphasis is also placed on addressing classical backtesting issues such as look-ahead bias, overfitting, and inaccurate modeling of operational rebalancing costs (*fee bleeding*). The proprietary Python framework (`portfolio_ml_v4.py`)[1] is built according to Object-Oriented Programming (OOP) principles and centralizes parameterization through a `dataclass Config`. This separation between configuration and execution logic ensures scalability and reproducibility of experiments.

4.3.1 Investable Universe and Survivorship-Bias Prevention

The system analyzes a heterogeneous universe of 48 assets, divided into three macro-categories (Equity, Bond, ETF). A noteworthy element is the intentional inclusion of so-called *distressed* or delisted assets (for example, the tickers `SIVBQ.PK` for Silicon Valley Bank or `CSGN.S` for Credit Suisse). Including failed securities is an empirical method to mitigate survivorship bias and to force models to learn the technical and macroeconomic patterns that precede catastrophic crashes.

4.3.2 Transaction-Cost Modeling

The impact of costs is handled granularly by assigning spreads in basis points (bps) proportional to the asset’s estimated liquidity. Values range from 5 bps for ultra-liquid ETFs (e.g., `SPY.P`) up to 200 bps for illiquid Over-The-Counter (OTC) equities. Costs are not applied to theoretical turnover, but rather to the *actual delta* between the drifted portfolio (altered by market movements) and the new target, thereby avoiding a systematic overestimation of operational frictions.

4.4 Data Processing and Feature Engineering

The quality of ML models depends on the feature space provided as input. The Feature Engineering module computes more than 50 indicators for each asset, structured into three main domains.

4.4.1 Technical and Momentum Domain

Prices are transformed into stationary metrics to facilitate learning:

- **Trend and Momentum:** Rolling returns over 5, 10, 20, and 60 days. Simple Moving Averages (SMA) are normalized as ratios relative to the current price, capturing deviations from the long-term mean (up to 200 days).

¹The concept of Risk Parity was formalized by Qian (2005), who showed that a portfolio in which each asset contributes equally to total risk produces a more robust diversification than simple equal-weighting of capital. See: E. Qian, “Risk Parity Portfolios”, *PanAgora Asset Management Research Paper*, 2005.

- **Oscillators:** The Relative Strength Index (RSI) and the Moving Average Convergence Divergence (MACD) provide signals about overbought/oversold conditions.
- **Volatility:** Bollinger Bands generate two features: the price position relative to the bands (%B) and band width (Bandwidth), an indicator of volatility expansion or contraction.

4.4.2 Macroeconomic Domain

The model is contextualized through macroeconomic proxies:

- The VIX index is used to determine the equity market *risk-on* or *risk-off* regime.
- The DXY (U.S. Dollar Index) and the 10-year Treasury yield curve (US10Y) help model inflationary pressures and monetary policy.

4.5 Formulation of the Dynamic Target

A classic problem in financial ML is defining the dependent variable y . Using a fixed threshold (e.g., +2% to classify an uptrend) is inadequate in a multi-asset portfolio: 2% may be statistical noise for a cryptocurrency, but an extreme move (a distribution tail event) for a short-term bond.

The system resolves this distortion by implementing **Dynamic Targeting** based on rolling volatility. The activation threshold for an asset i at time t is computed as:

$$\text{Threshold}_{i,t} = \sigma_{i,t}^{(20)} \times \sqrt{h} \times M \quad (1)$$

where $\sigma_{i,t}^{(20)}$ is daily volatility computed over the last 20 days, h is the forecasting horizon (`forward_period` = 10), and M is the multiplier (`target_std_multiplier` = 0.35).

The target is then discretized into three classes:

- +1 (*Bullish*): expected return > $\text{Threshold}_{i,t}$
- -1 (*Bearish*): expected return < $-\text{Threshold}_{i,t}$
- 0 (*Neutral*): sideways move within the threshold

This prevents permanent polarization of the model toward high-beta assets.

4.6 Predictive Engine, Hyperparameter Optimization, and Walk-Forward Validation

Learning is delegated to two state-of-the-art tree-based architectures: Random Forest (which reduces variance by aggregating uncorrelated trees) and XGBoost (which reduces bias by training sequential trees on the errors of previous ones). A third model, the Ensemble, combines the predicted probabilities from the first two to maximize robustness.

4.6.1 Hyperparameter Optimization

A critical aspect of the pipeline, absent from the original version, concerns the hyperparameter optimization procedure. If tuning were performed by looking at performance across the entire out-of-sample horizon, it would introduce a form of look-ahead bias at the model-selection level—exactly the bias that walk-forward validation aims to eliminate. To prevent this issue, the pipeline adopts a **triple-split** structure within each walk-forward window:

1. **Training set:** the first T_{train} days of the current window are used to train candidate models.
2. **Internal validation set:** the last $T_{\text{val}} = 5$ business days of the training window are reserved exclusively for *Bayesian Optimization* of hyperparameters via the `Optuna` library. The optimized parameters include:
 - `n_estimators` $\in [100, 800]$
 - `max_depth` $\in [3, 10]$
 - `learning_rate` $\in [0.01, 0.3]$ (XGBoost)
 - `min_samples_leaf` $\in [10, 100]$ (Random Forest)
 - `subsample` $\in [0.6, 1.0]$
3. **OOS test:** the following 21 business days constitute the uncontaminated out-of-sample test on which operational forecasts are produced.

The optimization metric on the validation set is **balanced accuracy**, chosen instead of standard accuracy to compensate for the natural class imbalance among the three target classes (Bullish, Neutral, Bearish), which varies dynamically with the market regime.

The triple-split structure ensures that the hyperparameter values chosen in each fold have never “seen” the OOS data on which the model is subsequently evaluated, thereby preserving the integrity of the empirical assessment.

4.6.2 Walk-Forward Methodology

Using k -fold cross-validation is flawed because it causes data leakage (mixing future and past data). The system adopts Walk-Forward validation with an expanding training window, structured as follows:

1. Train on the first 504 days (with the last 5 reserved for tuning, as described in §4.6.1).
2. Optimize hyperparameters on the internal validation set.
3. Produce out-of-sample (OOS) forecasts for the next 21 days (step size).
4. Move forward in time, incorporating the 21 days just elapsed into the new training set.

This approach, although computationally expensive, faithfully simulates the real operating conditions of a fund manager. The computational bottleneck was reduced by parallelizing computations through the `joblib` library, processing walk-forward runs for different assets across multiple cores and drastically reducing total runtime.

4.7 Backtesting Engine and Risk Parity

The final pipeline transforms signals into an asset allocation, addressing the fundamental problem of *fee bleeding*.

4.7.1 Risk-Parity Allocation

Instead of allocating capital equally among assets classified as “Bullish”, the algorithm distributes weight based on inverse volatility. The theoretical weight w_i of asset i is proportional to the score:

$$\text{Score}_i = \frac{\text{Signal}_i}{\sigma_{i,t}} \quad (2)$$

Weights are then normalized so that their sum is one:

$$w_i = \frac{\text{Score}_i}{\sum_{j \in \mathcal{A}_t} \text{Score}_j} \quad (3)$$

where $\mathcal{A}_t$ is the set of assets with an active signal at time t . This ensures that highly stable assets receive more capital than speculative assets, stabilizing the portfolio’s overall variance (an approach conceptually akin to Ray Dalio’s *All Weather* logic).

4.7.2 Drift and Rebalancing Management

Transaction costs are not naively applied to theoretical turnover, but to the actual delta between the drifted portfolio (altered by market movements) and the new target. Formally, the transaction cost at time t for asset i is:

$$C_{i,t} = \text{spread}_i \times |w_{i,t}^{\text{target}} - w_{i,t}^{\text{drifted}}| \quad (4)$$

Moreover, the engine executes trades only every 21 days. Trading daily (*daily rebalancing*) would have eroded almost all net profits due to micro-adjustment operations. By letting positions “run” for an entire lunar month, the strategy maximizes the compound annual growth rate (CAGR).

4.8 Evaluation Metrics and Graphical Output

The system is accompanied by a quantitative reporting suite that goes beyond absolute returns and evaluates the Risk–Return profile using academic measures. The main metrics are defined as follows.

Sharpe Ratio. It measures the average excess return over the risk-free rate r_f per unit of total volatility:

$$\text{SR} = \frac{\bar{r}_p - r_f}{\sigma_p} \quad (5)$$

where $\bar{r}_p$ is the portfolio’s annualized average return and σ_p is the annualized standard deviation of returns.

Sortino Ratio. It penalizes only negative volatility (*downside deviation* σ_d), making it more informative than the Sharpe Ratio alone for asymmetric distributions:

$$\text{SoR} = \frac{\bar{r}_p - r_f}{\sigma_d}, \quad \sigma_d = \sqrt{\frac{1}{T} \sum_{t=1}^T \min(r_t - r_f, 0)^2} \quad (6)$$

Maximum Drawdown. It measures the maximum peak-to-trough loss over the entire portfolio time series:

$$\text{MDD} = \min_{t \in [0, T]} \left(\frac{V_t - \max_{s \leq t} V_s}{\max_{s \leq t} V_s} \right) \quad (7)$$

where V_t is the portfolio value at time t .

Calmar Ratio. It relates the CAGR to the Maximum Drawdown in absolute value, summarizing the return earned per unit of maximum historical loss:

$$\text{CR} = \frac{\text{CAGR}}{|\text{MDD}|} \quad (8)$$

The framework also exports the following graphical outputs: a comparative Equity Curve, underwater drawdown charts, monthly return heatmaps, and an area allocation analysis through time.

Analysis and Discussion of Results

This chapter analyzes the empirical results obtained from the quantitative portfolio management system described in the previous chapter. The focus is primarily on the overall comparison between the three implemented Machine Learning models (XGBoost, Random Forest, and Ensemble) and the reference benchmarks, represented by the S&P 500 index and the static Fixed 40/30/30 strategy.

The analysis is developed through the main performance and risk metrics generated by the backtesting framework, with the aim of assessing the effectiveness of dynamic allocation based on predictive signals and risk-parity mechanisms.

The observed period covers 2016–2026, including different market regimes: the post-2016 bull phase, the 2020 pandemic crash, the 2022 bear market, and the subsequent recovery. The presence of profoundly different macroeconomic contexts makes the sample particularly useful for evaluating the system’s robustness.

5.1 Global Comparative Analysis

5.2 Equity Curve Comparison

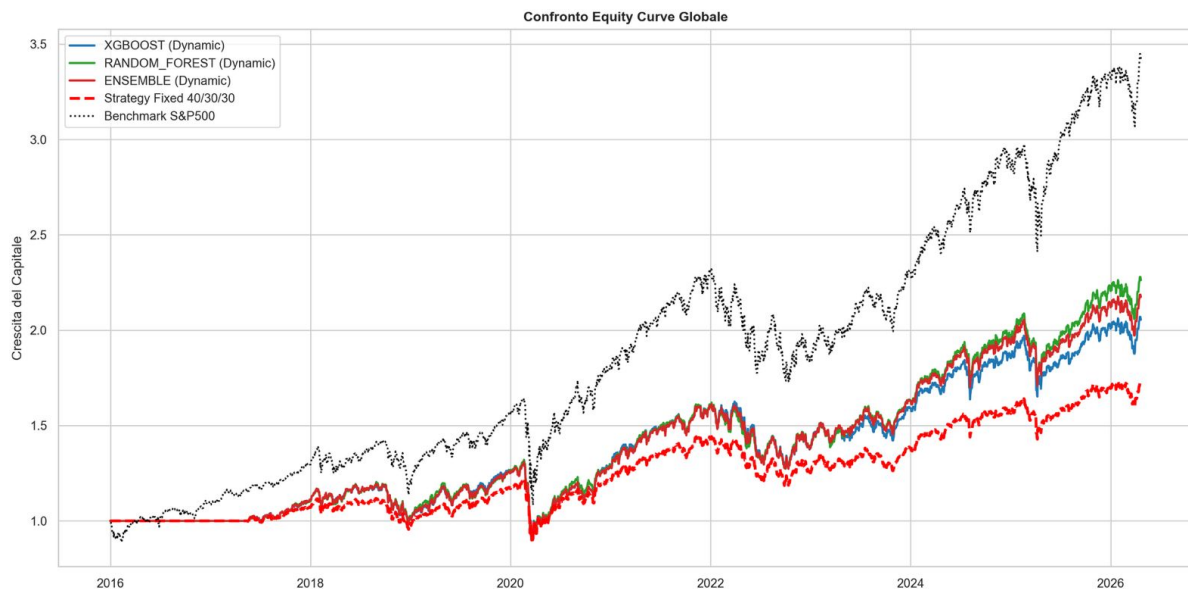


Figura 1: Equity-curve comparison over 2016–2026 for the three ML models with dynamic allocation (XGBoost, Random Forest, Ensemble), the Fixed 40/30/30 strategy, and the S&P 500 benchmark. Initial capital normalized to 1.

Figure 1 shows the evolution of invested capital across the strategies. The S&P 500 benchmark records the highest nominal growth, with a final multiplier of about 3,4 $\times$, but also exhibits the most pronounced volatility and drawdowns.

Among the ML models, Random Forest achieves the best result with a final capital growth of about 2,25 $\times$, followed by the Ensemble (2,18 $\times$) and XGBoost (2,05 $\times$). The Fixed 40/30/30 strategy confirms itself as the least performant, with final capital of about 1,7 $\times$.

During the 2020 pandemic crisis, ML models show a better ability to contain losses than the equity benchmark and recover pre-crisis levels more quickly. This dynamic is

consistent with the risk-parity reallocation mechanism, which automatically reduces the weight of assets characterized by rising volatility.

5.3 Comparative Annual Returns

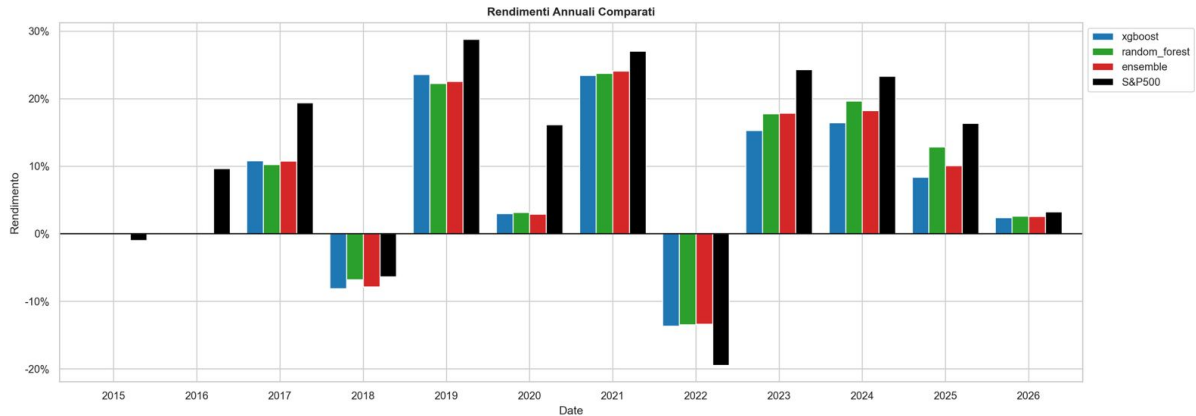


Figure 2: Annual returns of the three ML models and the S&P 500 over 2015–2026. Values are expressed as percentages.

Figure 2 allows one to observe the yearly behavior of the strategies across different market regimes.

In the initial biennium 2015–2016, the models do not produce operational output because the framework uses the first 504 trading days as the initial training window.

Over 2017–2019, ML models record positive and relatively stable returns. In 2018, the correction is more contained than for the equity benchmark, while in 2019 performance falls between +22% and +24%.

The year 2020 represents the most meaningful test for the framework: the models close the year in positive territory around +3%, absorbing the pandemic shock with lower volatility than the S&P 500.

In 2022, a year characterized by the bear market induced by rising interest rates, ML models limit losses to the range between −13% and −14%, versus −19% for the benchmark.

5.4 Risk–Return Profile

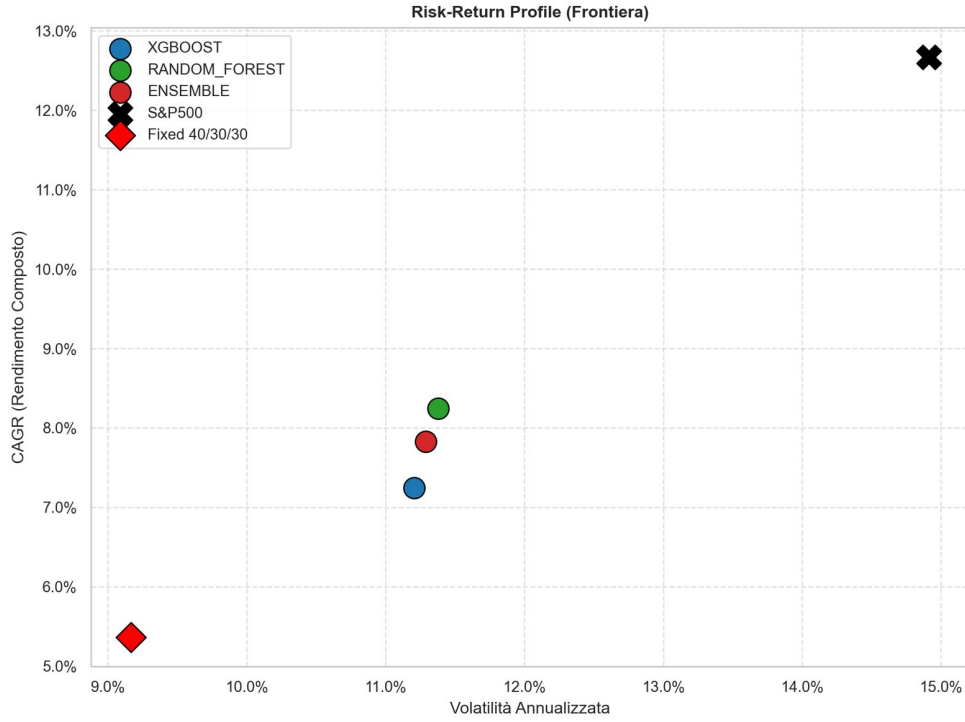


Figura 3: Positioning of strategies in the annualized-volatility–CAGR plane over 2016–2026.

Figure 3 summarizes the risk–return profile of the analyzed strategies.

Tabella 1: Summary of the main performance metrics over 2016–2026.

Strategy	CAGR (%)	Ann. Vol. (%)	Sharpe Ratio	Max DD (%)
XGBoost (Dynamic)	7,2	11,2	$\approx 0,65$	≈ -31
Ensemble (Dynamic)	7,8	11,3	$\approx 0,68$	≈ -31
Random Forest (Dynamic)	8,2	11,4	$\approx 0,73$	≈ -31
Fixed 40/30/30	5,4	9,1	$\approx 0,59$	≈ -22
S&P 500 (benchmark)	12,6	14,9	$\approx 0,85$	> -34

The three ML models show a very homogeneous profile, with annualized volatility close to 11% and CAGR between 7,2% and 8,2%. Random Forest is the model with the best risk–return trade-off within the ML cluster.

The S&P 500 maintains the highest absolute return, but at the cost of significantly higher volatility. The Fixed 40/30/30 strategy, instead, presents the most conservative profile, but also the most limited return.

5.5 Distribution of Daily Returns

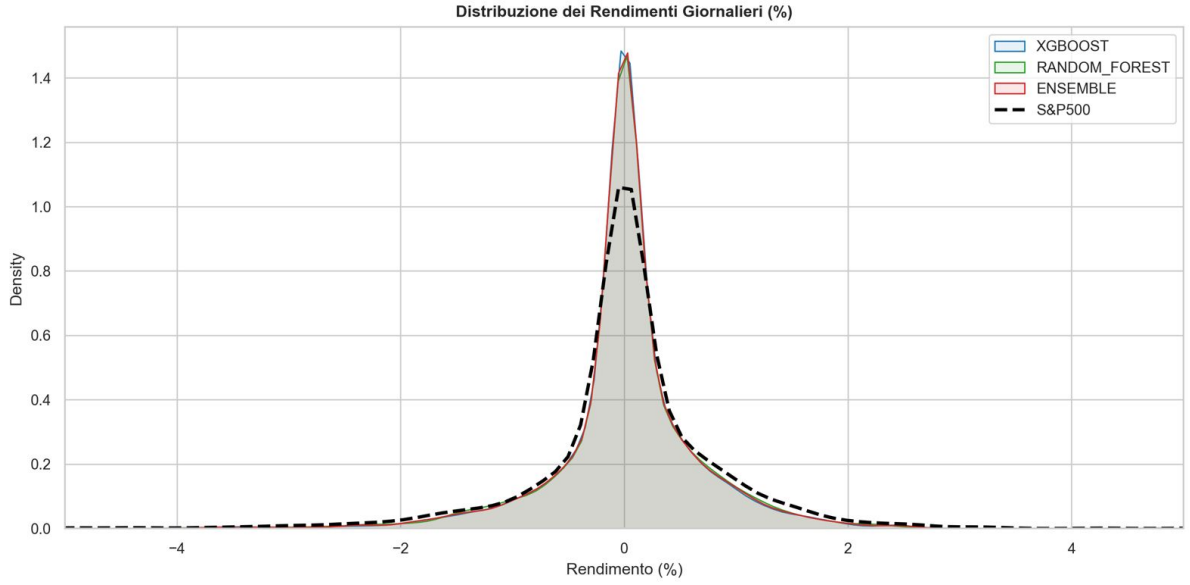


Figura 4: Density distribution of daily returns for the three ML models and the S&P 500.

Figure 4 compares the empirical distribution of daily strategy returns.

All distributions exhibit fatter tails than a normal distribution, a typical feature of financial series. However, ML models show greater central concentration and less extended left tails than the benchmark, highlighting lower exposure to extreme daily losses.

The strong overlap among the XGBoost, Random Forest, and Ensemble curves suggests that the risk-parity allocation component exerts a dominant influence on the return distribution relative to the predictive differences of individual algorithms.

6 Economic Interpretation of Results

The empirical analysis shows that the developed framework improves the risk–return profile relative to the static reference strategy, mainly through dynamic mechanisms for volatility control and capital reallocation.

Among the implemented models, Random Forest appears the most efficient in terms of the balance between annualized return and assumed risk, recording the best Sharpe Ratio and the highest CAGR within the ML cluster.

The analysis also suggests that the dynamic allocation component plays a central role in stabilizing the portfolio across different market regimes. In particular, features linked to volatility and the macro-financial context prove crucial in reducing exposure during phases of systemic stress.

The limited performance differences among XGBoost, Random Forest, and Ensemble instead suggest a high informational correlation among models, due to sharing the same feature space and the same risk-allocation logic.

7 Critical Discussion and Limitations

Although the results support the validity of the proposed framework, the work presents some methodological limitations typical of quantitative systems based on historical data.

First, backtesting cannot fully replicate real operating conditions in financial markets. Although walk-forward validation mechanisms, look-ahead bias controls, and transaction-cost modeling were implemented, the risk of overfitting and data-mining bias remains.

Second, the historical sample analyzed, while including different market regimes, does not include extreme systemic events such as the 2008–2009 financial crisis. Extending the dataset further back in time could therefore strengthen the statistical robustness of the conclusions.

Additional limitations concern the strong correlation among the implemented models, which reduces the effectiveness of the Ensemble, and the absence of full modeling of phenomena such as market impact, sudden liquidity changes, and execution slippage.

Overall, the framework shows that integrating Machine Learning with dynamic risk allocation can provide valuable support to quantitative portfolio management, while still requiring further robustness checks and out-of-sample generalization tests.

Conclusions

This dissertation analyzed the application of Machine Learning models to quantitative portfolio management, integrating tree-based predictive algorithms with a dynamic allocation framework grounded in risk-parity logic. The aim of the research was to verify whether the use of non-linear predictive signals could improve the efficiency of the risk–return profile relative to traditional static strategies.

The empirical results show that integrating the predictive component with dynamic risk control makes it possible to build portfolios that are more resilient across different market regimes. In particular, the system demonstrated good adaptability during the main stress phases of the analyzed period, containing volatility and drawdowns relative to a pure equity benchmark.

The comparative analysis among XGBoost, Random Forest, and Ensemble also highlights how the asset-allocation component plays a predominant role relative to marginal differences among individual predictive algorithms. The results therefore suggest that, within multi-asset quantitative architectures, dynamic risk management is as decisive a factor as the quality of the forecasting model itself.

From a methodological perspective, the work confirms the growing relevance of Machine Learning techniques in quantitative asset management. However, it also underscores the need to interpret these tools not as systems capable of eliminating market risk, but as mechanisms that can improve its management through adaptive, data-driven approaches.

Some structural limitations remain. The framework was validated exclusively through historical backtesting and does not fully account for phenomena such as market impact, liquidity variations, or extreme structural changes in financial markets. Moreover, the high correlation among implemented models limits the benefits derived from building the Ensemble.

Future developments could include integrating more heterogeneous architectures, such as recurrent neural networks, transformer models, or reinforcement-learning approaches, as well as extending the framework to different macroeconomic scenarios and broader investment universes.

Overall, the work demonstrates that the combined use of Machine Learning and dynamic risk allocation is a practically applicable approach to quantitative portfolio management, capable of improving the stability and efficiency of the decision-making process in financial contexts characterized by high complexity and uncertainty.

Riferimenti bibliografici

- [1] D. De Prosperis et al. *ML Portfolio Backtester*. <https://github.com/deprosperisdomenico/ml-portfolio-backtester>. GitHub repository. 2026.
- [2] E. F. Fama e K. R. French. “Common risk factors in the returns on stocks and bonds”. In: *Journal of Financial Economics* 33.1 (1993), pp. 3–56.
- [3] S. Gu, B. Kelly e D. Xiu. “Empirical asset pricing via machine learning”. In: *Review of Financial Studies* 33.5 (2020), pp. 2223–2273.
- [4] C. Krauss, X. A. Do e N. Huck. “Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500”. In: *European Journal of Operational Research* 259.2 (2017), pp. 689–702.
- [5] H. Markowitz. “Portfolio selection”. In: *Journal of Finance* 7.1 (1952), pp. 77–91.